

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 6

**Настройка CI/CD для автоматического развёртывания
backend-приложения**

Выполнил:
Цатинян Артём
Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

В рамках домашнего задания требовалось настроить автоматическое развёртывание backend-приложения на удалённый сервер с использованием GitHub Actions или другой CI/CD-системы. Автодеплой должен запускаться при обновлении кода в выбранной ветке репозитория.

2 Ход работы

2.1 Выбор подхода

Для CI/CD была выбрана GitHub Actions, так как проект хранится в GitHub-репозитории. Развёртываемое приложение — микросервисная версия сервиса бронирования ресторанов из ЛР2-ЛР4.

Во время ручного деплоя было выявлено, что сборка Gradle на сервере с 2 GB RAM работает слишком медленно и создаёт высокую нагрузку. Поэтому pipeline построен так, чтобы тяжёлая часть выполнялась на GitHub runner:

1. GitHub Actions получает код репозитория.
2. На runner устанавливается Java 21.
3. Выполняется сборка Spring Boot jar-файлов.
4. Готовые jar-файлы архивируются.
5. Архив передаётся на сервер по SSH/SCP.
6. На сервере архив распаковывается, после чего systemd-сервисы перезапускаются.

Такой подход экономит ресурсы VPS: сервер не компилирует Kotlin-проект, а только запускает готовые jar-файлы.

2.2 Workflow GitHub Actions

Файл workflow добавлен по пути:

```
.github/workflows/deploy-lab2.yml
```

Workflow запускается при push в ветку `lab4-dz6`, а также вручную через `workflow_dispatch`.

```
on:
  push:
    branches:
      - lab4-dz6
  workflow_dispatch:
```

2.3 Сборка приложения

В CI используется Java 21:

```
- name: Set up Java
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: "21"
```

Далее выполняется сборка:

В workflow рабочей директорией указан каталог `labs/lab2` внутри папки студента. Команда сборки:

```
chmod +x ./gradlew
./gradlew --console=plain bootJar
```

После сборки создаётся архив с нужными артефактами:

```
tar -czf lab2-jars.tar.gz \
  deploy-systemd.sh \
  identity-service/build/libs/identity-service.jar \
  catalog-service/build/libs/catalog-service.jar \
  booking-service/build/libs/booking-service.jar \
  review-service/build/libs/review-service.jar
```

2.4 Передача артефактов на сервер

Для загрузки архива используется action `appleboy/scp-action`. Он подключается к серверу по SSH и кладёт архив в `/tmp/lab2-jars.tar.gz`.

Для подключения используются secrets репозитория:

Secret	Назначение
DEPLOY_HOST	IP-адрес сервера: 186.246.13.204
DEPLOY_USER	Пользователь SSH: root
DEPLOY_PASSWORD	Пароль SSH пользователя
DEPLOY_PORT	SSH-порт, необязательный secret, по умолчанию 22
DEPLOY_PATH	Путь установки, необязательный secret, по умолчанию <code>/opt/restaurant-booking-lab2</code>

2.5 Перезапуск сервисов

После загрузки артефактов запускается SSH-шаг:

```
mkdir -p "$APP_DIR"
tar -xzf /tmp/lab2-jars.tar.gz -C "$APP_DIR"
cd "$APP_DIR"
sed -i 's/\r$//' deploy-systemd.sh
chmod +x deploy-systemd.sh
./deploy-systemd.sh
```

Скрипт `deploy-systemd.sh` создаёт и обновляет unit-файлы `systemd` для четырёх сервисов:

Systemd unit	Сервис
<code>restaurant-booking-identity.service</code>	<code>identity-service</code>
<code>restaurant-booking-catalog.service</code>	<code>catalog-service</code>
<code>restaurant-booking-booking.service</code>	<code>booking-service</code>
<code>restaurant-booking-review.service</code>	<code>review-service</code>

Сервисы включаются в автозагрузку и перезапускаются:

```
systemctl daemon-reload
systemctl enable restaurant-booking-identity restaurant-booking-catalog
    restaurant-booking-booking restaurant-booking-review
systemctl restart restaurant-booking-identity
systemctl restart restaurant-booking-catalog
systemctl restart restaurant-booking-booking
systemctl restart restaurant-booking-review
```

2.6 Проверка после деплоя

В конце workflow выполняется smoke-check OpenAPI endpoints:

```
curl -fsS http://localhost:8081/v3/api-docs >/dev/null
curl -fsS http://localhost:8082/v3/api-docs >/dev/null
curl -fsS http://localhost:8083/v3/api-docs >/dev/null
curl -fsS http://localhost:8084/v3/api-docs >/dev/null
```

Если хотя бы один сервис не запустился, команда `curl -fsS` завершится с ошибкой, и GitHub Actions пометит деплой как неуспешный.

2.7 Результат

После настройки сервер содержит рабочую ручную инсталляцию приложения, а pipeline автоматизирует повторный деплой при обновлении ветки `lab4-dz6`. Сервисы доступны по публичным адресам:

Сервис	Swagger UI
identity-service	https://ezhidze.ru/identity/swagger-ui/index.html
catalog-service	https://ezhidze.ru/catalog/swagger-ui/index.html
booking-service	https://ezhidze.ru/booking/swagger-ui/index.html
review-service	https://ezhidze.ru/review/swagger-ui/index.html

3 Вывод

В рамках домашнего задания был настроен CI/CD pipeline на GitHub Actions. Pipeline автоматически собирает микросервисное backend-приложение, передаёт готовые jar-файлы на сервер и перезапускает systemd-сервисы. Такой вариант учитывает ограниченные ресурсы VPS и переносит тяжёлую сборку на GitHub runner.

В результате обновление ветки `lab4-dz6` может автоматически приводить к обновлению работающего приложения на удалённом сервере.